Work Assignment
Phase 1

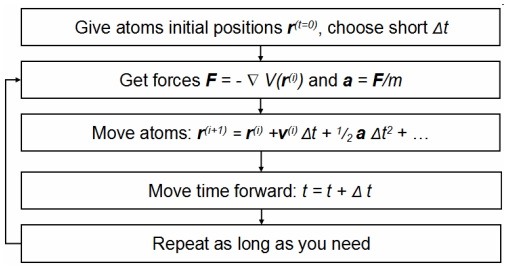
Learning outcomes

Students should become aware of the case study specificities: i) the time profile of the several functions; ii) dependency constraints among function calls; iii) main opportunities to explore parallelism. Students should also show a basic understanding of how to explore parallelism with OpenMP, including scalability analysis, how to manage dependency constrains (including data races) and identify performance tradeoffs.

Introduction

The program to analyse and optimise is one simple molecular dynamics' simulation code applied to atoms of argon gas. The original code was taken from *FoleyLab/MolecularDynamics: Simple Molecular Dynamics*, slightly modified for this course context. On this first phase, students are requested to develop an OpenMP parallel version of the code supplied in the course e-Learning page.

The code follows a generic approach to simulate particle movements over time steps, using the Newton law. It uses Lennard Jones potential to describe the interactions among two particles (force/potential energy). The Verlet integration method is used to calculate the particles trajectories over time. Boundary conditions (i.e., particles moving outside the simulation space) are managed by elastic walls (see supporting slides on the e-Learning page)



In this assignment, students should follow a structured methodology to develop parallel programs, covering the following steps:

- **(i)** **identify** the application hot-spots (code blocks with high computation time)[1];
- **(ii)** **analyse** and present the alternatives to explore parallelism within the hot-spots identified in **(i)**;
- **(iii)** **select** an approach to explore parallelism, justified by a scalability analysis;
- **(iv)** **implement** and **optimise** the approach on the Deucalion cluster (compute nodes on the `arm` queues);
- **(v)** **measure** and **discuss** the performance of the proposed solution.

Groups, submission format and dates

The work assignment is to be performed by a group of two students.

The work must be submitted through the e-learning platform, **on a zip file** that, when unzipped, should generate a base directory whose name is the group elements, e.g., pg43000_pg54000. It should include:

- a 2-page PDF report (annexes excluded, these might be read by the evaluator) using the IEEE template (in https://www.ieee.org/conferences/publishing/templates.html);
- a subdirectory with all source code (please, do not submit executables **or other files**);
- a **single** `Makefile` that generates an executable named `md` in the base directory (please, follow the `Makefile` template given in the annex B). The `Makefile` must include a target `run` that generates and executes parallel version of the code (i.e., by executing `make run`, see example in annex).

Submission deadline: 23:59, 25-Nov-26. The defence of this assignment will be performed during the oral presentation of the WA-Phase 2 (in Jan'27).

[1]Initial profile (before any optimization) can be done on a simulation with a lower number of particles (e.g, N=200 in line 214)

Evaluation

The evaluation of this work will consider:

- **(i)** the **execution time** of the OpenMP implementation on Deucalion compute nodes (`arm` queues); the best number of PUs can be specified in the `Makefile` **(30%);**
- **(ii)** the selected **approach to explore parallelism**, its **implementation** with OpenMP and **code legibility (30%); note**: projects with data races will have a **strong** penalty
- **(iii)** the **report quality,** including **strong scalability analysis**, profiling and other models and metrics that explain the results **(40%).**

Annex A – Instructions to extend the provided *Makefile*

The work submission must include **single** `Makefile` that generates one executable `md` in the base directory. All source files should be placed in a subdirectory (e.g. `src`).

The program will be run for testing using `make run` and to take the parallel execution time. Any necessary parameters, environment variables, etc., should be specified to minimize the program's execution time.

Suggestions/notes:
1. Ensure that the parallel version uses the number of threads that minimizes execution time, by exporting the value of the variable `OMP_NUM_THREADS` inside the `Makefile` , as in the following example (note: the operator `?=` assigns a value if it was not defined before; this make it possible to override the value specified inside the `Makefile` by exporting a different one before `make run`):

```
        ...
export OMP_NUM_THREADS ?= 48
run:
        ./md < inputdata.txt
```

2. The `Makefile` provided is a simplification of the one in the original repository. Any additional compilation flags can be used: `-fopenmp` is mandatory to enable OpenMP; `-march=native` is suggested for better tunning to the target ARM machine; `-g` and `-fno-omit-frame-pointer` can help to give more accurate results on some profiles (e.g., `perf`).

3. The execution time will be taken by compiling the program with GCC/15.2.0 on an ARM node. Students can fill `CFLAGS` with any flags supported by this compiler.

Annex B - The simple *Makefile* provided

```
CC       = gcc
SRC      = src/
CFLAGS   = # none

.DEFAULT_GOAL = md

md: $(SRC)MD.cpp
        $(CC) $(CFLAGS) $(SRC)MD.cpp -lm -o md

clean:
        rm ./md

export OMP_NUM_THREADS ?= 48
run:
        ./md < inputdata.txt
```